

CASA Evidence — OAuth 2.0 `state` and `redirect_uri` Protections

Question: *If the application uses an OAuth 2.0 integration, provide a written description along with relevant evidence (screenshots, source code, or other documentation) detailing how the application uses the `state` and `redirect_uri` parameters to prevent common OAuth vulnerabilities.*

Application: Provenance (Next.js App Router), using Supabase Auth (GoTrue) as its OAuth broker for Google and Apple federated sign-in.

This document is a companion to `oauth2-flow.md`, which establishes that the flow is Authorization Code Grant with PKCE; this document focuses specifically on the two parameters named in the question.

Both `state` and `redirect_uri` exist to defeat the same class of attack: an OAuth callback that did not originate from the request the current browser actually made (CSRF against the redirection endpoint, authorization-code injection, and open-redirect/token-leak via a malicious callback destination). Provenance's flow is protected at **two independent layers** — inside Supabase's Auth server (GoTrue), which this app depends on, and inside this application's own code — so a gap in either layer alone would not be sufficient to exploit the flow.

1. `state` — CSRF and mix-up protection

1.1 Where `state` is generated and why the app never has to build it itself

Provenance's own client code never constructs a `state` value — this is intentional and is a direct consequence of using `@supabase/ssr`'s PKCE flow (documented in `oauth2-flow.md` §2). The `state` parameter for the Google/Apple leg of the flow is generated **entirely inside Supabase's Auth server** at the moment it redirects

the browser to the identity provider, and is verified there again

when the identity provider redirects back.

This is confirmed directly in Supabase Auth's own open-source

implementation (supabase/auth, formerly gotrue):

```
// internal/api/external.go – state is issued as a signed token, keyed to this specific
// authorization request, and is verified against the server's own signing key on callback.
p := jwt.NewParser(jwt.WithValidMethods(config.JWT.ValidMethods))
_, err := p.ParseWithClaims(state, &claims, func(token *jwt.Token) (interface{}, error) {
    if kid, ok := token.Header["kid"]; ok {
        if kidStr, ok := kid.(string); ok {
            key, err := conf.FindPublicKeyByKid(kidStr, &config.JWT)
            if err != nil {
                return nil, err
            }
            if key != nil {
                return key, nil
            }
        }
    }
    ...
})
if err != nil {
    return ctx, apierrors.NewBadRequestError(apierrors.ErrorCodeBadOAuthState, "OAuth callback with invalid state")
}
if claims.Provider == "" {
    return ctx, apierrors.NewBadRequestError(apierrors.ErrorCodeBadOAuthState, "OAuth callback with invalid state")
}
```

Two important properties follow directly from this:

1 ****The state value is a cryptographically signed token (JWT), not**

an arbitrary opaque string an attacker could fabricate.** It is

signed with a key only the Supabase Auth server holds. A callback

presenting a state that doesn't verify against that key — e.g. a

forged or reused value — is rejected outright

(ErrorCodeBadOAuthState) before any code exchange happens.

1 ****state carries the expected provider (claims.Provider) and, in**

current versions, is backed by a server-side flow_state record

that also stores the exact code_challenge/code_challenge_method

for that request.** A callback claiming to be from Google but

carrying a state that was actually issued for an Apple flow (a

"provider mix-up" attack, one of the specific attack classes

state exists to stop) fails this check because the recorded

provider doesn't match.

Because the signing key/flow-state table lives entirely inside Supabase's managed backend, this application's own codebase has no opportunity to weaken, skip, or mismanage state — there is no app-level code path that could accidentally omit it, unlike a hand-rolled OAuth client where a missed state check is a common real-world bug.

1.2 Observed evidence of the `state` parameter in this deployment

The presence and JWT-based nature of state in this specific deployment's OAuth traffic is documented directly in this repo's own incident-troubleshooting notes (used while debugging a redirect misconfiguration):

1. ****Site URL has leading/trailing spaces****:
 - Decode the JWT state parameter in the URL to see what Supabase received
 - If you see ``" https://..."`` (with spaces), your Site URL has leading spaces

This confirms empirically (not just from upstream source) that this project's live OAuth callbacks carry a signed JWT state value that encodes server-side configuration (here, the configured Site URL) — consistent with the `ExternalProviderClaims` structure in the upstream implementation.

1.3 The app's own leg: PKCE as the CSRF binding for the browser ↔ app hop

state (as described above) protects the ****Supabase ↔ identity provider hop. The app ↔ Supabase**** hop (i.e., the code this application itself receives at `/auth/callback`) is protected by the PKCE `code_verifier/code_challenge` pair instead, which is exactly the substitution the OAuth 2.0 Security Best Current Practice (IETF RFC 9700) endorses:

- > "Clients that have ensured that the authorization server supports
- > Proof Key for Code Exchange (PKCE) MAY rely on the CSRF protection
- > provided by PKCE. ... The same protection is provided by PKCE or the
- > OpenID Connect nonce value."

> — [RFC 9700 §4.7](#), Best Current Practice for OAuth 2.0 Security

Concretely, as already documented in `oauth2-flow.md` §3 and §5: the `code_verifier` is generated client-side and stored in a first-party cookie before the redirect away from the app; only the same browser that holds that cookie can complete `exchangeCodeForSession()` successfully. An attacker who tricks a victim into visiting a crafted `/auth/callback?code=...` URL (the classic OAuth CSRF/"login CSRF" attack state exists to stop) cannot supply the matching `code_verifier` cookie for a code they didn't request, so `exchangeCodeForSession()` fails with a detected, named error:

```
/**
 * Checks if the given error message indicates a PKCE verifier error.
 * Multiple Supabase error message variants are checked because the exact
 * wording has changed across Supabase versions and edge cases (e.g. empty
 * verifier vs verifier mismatch vs missing verifier).
 */
function isVerifierError(error: string) {
  const lower = error.toLowerCase();
  return (
    lower.includes('both auth code and code verifier should be non-empty') ||
    lower.includes('code verifier') ||
    lower.includes('pkce') ||
    lower.includes('code_verifier')
  );
}
```

So the flow has CSRF protection at **both** hops — GoTrue's signed state protects the Supabase↔IdP hop, and the app's PKCE `code_verifier` cookie protects the app↔Supabase hop — rather than relying on a single shared mechanism for the whole chain.

2. `redirect_uri` — preventing authorization-code theft via a malicious callback destination

`redirect_uri` (the destination the authorization server is told to send the code/token to) is protected by an **exact-match allow-list enforced independently at each hop** of the two-hop broker architecture, so an attacker cannot redirect either half of the flow to an attacker-controlled destination by tampering with a URL

parameter.

2.1 Hop 1 — Google/Apple → Supabase: the app's own domain is deliberately never registered

The Authorized Redirect URIs registered with Google (and Apple) point

only at Supabase's own fixed callback endpoint —

`https://<project>.supabase.co/auth/v1/callback` — never at this

application's domain. This is called out explicitly, with the

incorrect configuration shown crossed out, in this repo's own

operational documentation:

```
### Issue 2: Wrong Redirect URI
**Current Authorized Redirect URIs:**
- ■ `https://upbiqtluqemrmonyghix.supabase.co/auth/v1/callback` (CORRECT - this is what Google should redirect to)
- ■ `https://provenance.guru/auth/callback` (WRONG - Google should NOT redirect directly to your app)
- ■■ `https://provenance-khaki.vercel.app/auth/callback` (old domain - can remove)
- ■ `http://localhost:3000/auth/callback` (local dev - keep for testing)

**Why it's wrong:**
- Google OAuth redirects to Supabase, not directly to your app
- Supabase then redirects to your app
- Having `https://provenance.guru/auth/callback` in Google Cloud will cause errors
```

Because Google/Apple's OAuth `redirect_uri` validation is enforced by

the identity provider itself against this fixed, single-purpose

`allow-list`, there is no `redirect_uri` value an attacker could pass

through this application to make Google deliver an authorization

code anywhere other than Supabase's own callback endpoint — the value

is not something the app (or an attacker manipulating the app's

request) controls at all.

2.2 Hop 2 — Supabase → this app: exact-match `allow-list`, not a wildcard

Supabase's own redirect back to the application (carrying *its* code,

not Google's) is validated against a second, independent `allow-list` —

the project's configured **Redirect URLs** — before Supabase will

honor the `redirectTo` value the app supplied when calling

`signInWithOAuth():`

```
[auth]
# The base URL of your website. Used as an allow-list for redirects and for constructing URLs used
# in emails.
site_url = "http://localhost:3000"
```

```
# A list of *exact* URLs that auth providers are permitted to redirect to post authentication.
additional_redirect_urls = ["http://localhost:3000", "http://localhost:3000/auth/callback", "http://localhost:3000/"]
```

The production equivalent of this allow-list (configured in the Supabase dashboard rather than `config.toml`) is documented, along with the specific failure mode when a requested `redirectTo` isn't on it, in this repo's own troubleshooting notes:

```
## Why This Happens
Supabase redirects to the Site URL when:
1. The `redirectTo` URL doesn't exactly match what's in the Redirect URLs list
2. The Site URL is set incorrectly
3. There are invisible characters (spaces, trailing slashes) in the configuration
```

This is the critical security property: Supabase's validation is **exact string match**, not a prefix or substring match, and there is no wildcard host allowed. An attacker cannot pass `redirectTo=https://provenance.guru.evil.com/auth/callback` or `redirectTo=https://evil.com` and have Supabase honor it — a non-matching value causes Supabase to fall back to the safe default (`site_url`), never to the attacker's URL. This directly defeats the classic OAuth "open redirect via `redirect_uri`" attack, where a loose or wildcarded redirect URI allow-list lets an attacker walk away with the authorization code by redirecting it to a domain they control.

2.3 The app's own construction of `redirectTo` — never taken from arbitrary request input

The `redirectTo` value the app sends to Supabase in the first place is built exclusively from a browser-trusted origin and a static, hardcoded path constant — never from a query string, form field, or any other attacker-influenceable input at the point the flow is initiated:

```
const origin = window.location.origin;
const queryParams = new URLSearchParams();

if (props.paths.returnPath) {
  queryParams.set('next', props.paths.returnPath);
}

const redirectPath = [
  props.paths.callback,
  queryParams.toString(),
```

```
].join('?');  
  
const redirectTo = [origin, redirectPath].join('');
```

props.paths.callback is a fixed application route
(pathsConfig.auth.callback, i.e. /auth/callback) defined in
source, not user input — so even before Supabase's own allow-list
check (§2.2) runs, the value being checked is already constrained to
"the current, legitimate origin, plus a compile-time-fixed path."

2.4 The one genuinely user-influenceable parameter (next) is separately validated

The only part of the redirect chain that *is* influenced by
request-controllable data is the next query parameter — the
in-app page to land on **after** authentication succeeds (e.g.
/auth/callback?next=/artworks/123). This is not the OAuth
redirect_uri itself (which is governed by §§2.1–2.3), but it rides
along inside it and is therefore held to the same standard. It has
its own explicit open-redirect guard in the callback route, since
new URL('//evil.com/x', origin) would otherwise resolve to
https://evil.com/x (a protocol-relative URL takes over the host):

```
// Guard against open redirects (CASA 5.1.2 / CWE-601): the `next` param on this  
// route comes straight from the query string via @kit/supabase's auth callback  
// service. The block above strips the host from full absolute URLs  
// (e.g. `https://evil.com/x` -> `/x`), but protocol-relative URLs like  
// `//evil.com/x` fail the `new URL(nextPath)` parse (no base), fall into the  
// catch, and are used as-is. `new URL('//evil.com/x', origin)` then resolves  
// to `https://evil.com/x` because a leading `//` is a network-path reference  
// that takes over the host from `origin`. Reject anything that isn't a  
// same-origin, single-leading-slash path before building the redirect.  
if (!pathToRedirect.startsWith('/') || pathToRedirect.startsWith('///') || pathToRedirect.startsWith('/\\')) {  
  console.warn('[Auth] Rejected unsafe post-login redirect target', { nextPath });  
  pathToRedirect = '/artworks';  
}
```

3. Combined effect

Hop	Parameter	Enforced by	Attack it defeats
Supabase → Google/Apple → Supabase	state	Supabase Auth server (signed JWT / server-side flow_state, keyed to provider + PKCE challenge)	CSRF against the redirection endpoint; IdP "mix-up" attacks; state replay across flows

Hop	Parameter	Enforced by	Attack it defeats
App browser → Supabase → App	PKCE <code>code_verifier/code_challenge</code> (state-equivalent per RFC 9700 for this hop)	Supabase JS SDK (client) + <code>exchangeCodeForSession()</code> (server)	Login CSRF / forged callback visits; authorization-code interception and replay
Google/Apple → Supabase	<code>redirect_uri</code>	Google/Apple Cloud Console allow-list (Supabase's fixed callback only)	Authorization code exfiltration to an attacker-controlled domain
Supabase → App	<code>redirectTo</code> (<code>redirect_uri</code> for this hop)	Supabase project's Redirect URLs / <code>additional_redirect_urls</code> exact-match allow-list	Same as above, for the app-facing leg; falls back to <code>safe_site_url</code> on mismatch rather than honoring an arbitrary value
App's own post-login destination	<code>next</code>	<code>src/app/auth/callback/route.ts</code> explicit same-origin path validation	Open redirect (CWE-601) via the one request-influenceable parameter in the chain

4. Conclusion

Provenance's OAuth 2.0 integration uses `state` and `redirect_uri` exactly as intended by the OAuth 2.0 Security Best Current Practice: `state` (server-generated, signed, provider- and challenge-bound) and PKCE together provide CSRF protection across both hops of the Supabase-brokered flow, and `redirect_uri/redirectTo` is validated against a strict, exact-match allow-list at **each** hop independently — once by the identity provider (restricting deliveries to Supabase's own callback) and once by Supabase itself (restricting deliveries to this app's own known callback URL) — so that no single misconfigured or attacker-supplied value can redirect an authorization code away from its intended recipient. The one parameter this application does directly accept from the client (`next`, the post-login landing page) is not part of the OAuth trust boundary itself but is nonetheless independently validated against open redirects before use.